Overview of Application Functions, Methods & Operations

This technical specification lists the backend classes, frontend components, methods, event listeners, and configuration profiles comprising the Chat-App project. It outlines how components communicate via WebSockets (STOMP) and REST endpoints.

1. Backend Controllers

AuthController

Purpose: Manages session workflows, account registrations, and user identity validation.

`POST /api/auth/login`
Authenticates login credentials via standard security managers, registers user context, and builds a JWT security payload containing username, ID, and theme configurations.

`POST /api/auth/register`
Registers a new user. Performs username availability validations, hashes passwords with BCrypt, assigns a default avatar style, and persists the entity.

`GET /api/auth/me`
Fetches basic details of the currently authenticated principal to reconstruct authorization contexts on frontend page mounts.

ChatController

Purpose: Processes real-time messaging payloads and WebSocket session events.

`STOMP /app/chat/{roomId}`
Handles user-to-server chat message payload mappings. Validates the principal context, persists messages through service methods, and broadcasts updates to room subscribers.

`EventListener SessionConnectEvent`
Triggers when a client binds a WebSocket connection. Sets their active flag to true in the database and broadcasts their presence globally.

`EventListener SessionDisconnectEvent`
Triggers when a WebSocket connection drops. Sets user availability flags to false and alerts other active members via global presence topics.

RoomController

Purpose: Exposes rest methods to query, select, and build chat channels.

`GET /api/rooms`
Fetches all active rooms configured in the database to display in sidebar lists.

`POST /api/rooms`
Creates a new room. Throws bad request alerts if a channel name conflicts with an existing record.

`GET /api/rooms/{roomId}/messages`
Fetches all saved text messages associated with a room, sorted in ascending chronological order.

UserController

Purpose: Provides identity mapping endpoints.

`GET /api/users`
Returns a list of all registered accounts. Maps models into secure DTO objects to filter out private credential data.

`GET /api/users/online`
Returns a filtered list of users whose current availability states are online.

2. Backend Services

UserService

Purpose: Transactional handler for user accounts and presence updates.

`findByUsername(String username)`
Queries a user record by name. Returns an Optional wrapper to safely prevent NullPointer exceptions.

`getOnlineUsers()`
Queries all user database records whose online availability flag is active.

`setUserOnlineStatus(String username, boolean isOnline)`
Updates the online status flag of a user and logs the updated timestamp before saving the record.

RoomService

Purpose: Handles room creation and queries.

`getAllRooms() / getRoomById(Long id) / getRoomByName(String name)`
Queries the room database repository to retrieve matching configurations.

`createRoom(String name, String description)`
Creates a new ChatRoom record, checking for name duplicates before saving.

MessageService

Purpose: Processes incoming messages.

`saveMessage(String username, Long roomId, String content)`
Resolves the user and room entities from databases, builds a Message object, sets the timestamp to now, and saves it.

`getMessagesByRoom(Long roomId)`
Queries chronological messages mapped to a room ID.

3. Security & Socket Configurations

WebSocketConfig

Purpose: Configures the socket parameters and connection handshake authentication.

`registerStompEndpoints(StompEndpointRegistry registry)`
Exposes the "/ws" endpoint with SockJS fallback support and binds a HandshakeInterceptor to parse query parameters (e.g. "?token=...") during connection requests.

`configureMessageBroker(MessageBrokerRegistry registry)`
Sets up a simple built-in message broker mapping "/topic" for broadcasts and "/app" for client destination prefixes.

`configureClientInboundChannel(ChannelRegistration registration)`
Hooks a ChannelInterceptor to read WebSocket headers, extract JWT tokens, validate signatures, and bind the user security context principal to incoming messages.

WebSecurityConfig

Purpose: Spring Security profile settings.

`securityFilterChain(HttpSecurity http)`
Declares stateless sessions, bypasses authentication checks on "/api/auth/**" and "/error", manages CORS filters, and sets the entry point for unauthorized requests.

4. Frontend Utilities & Components

Login (Component)

Purpose: Auth capture login form.

`handleSubmit(e)`
Intercepts form submissions, validates that required fields are populated, and fires the onLogin context callback.

Register (Component)

Purpose: Handles user registrations.

`handleSubmit(e)`
Validates details, enforces a minimum password length, passes color configurations, and runs the registration callback.

ChatRoom (Component)

Purpose: The workspace hub connecting channels, presence lists, and real-time feeds.

`fetchData (Effect)`
Invoked on layout mount to load configured channels and users lists from the API.

`WebSocket Connection (Effect)`
Instantiates a STOMP client using a SockJS fallback, registers credentials, maps connectivity states, and handles presence updates.

`Room Subscription (Effect)`
Triggers when a channel is selected. Clears active arrays, fetches history, and subscribes to "/topic/room/{id}".

`handleSendMessage(e)`
Publishes message strings to "/app/chat/{roomId}" and clears input fields.

`handleCreateRoom(e)`
Submits new channel requests to the backend API and focuses the created room.

`handleLeaveRoom()`
Clears the selected room state, automatically triggering the WebSocket unsubscription.